

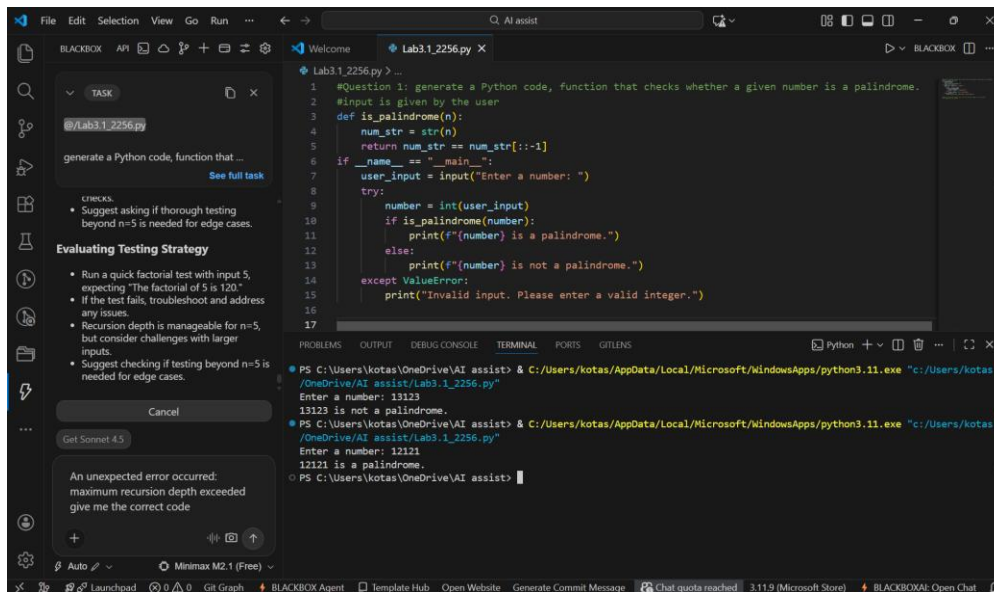
Lab Assignment- 3.1

AI Assisted Coding

R.Pranitha – 2303A52248

Question1: Zero-Shot Prompting (Palindrome Number Program)

Prompt: generate a Python code, function that checks whether a given number is a palindrome. And input is given by the user



```
1 #Question 1: generate a Python code, function that checks whether a given number is a palindrome.
2 #input is given by the user
3 def is_palindrome(n):
4     num_str = str(n)
5     return num_str == num_str[::-1]
6 if __name__ == "__main__":
7     user_input = input("Enter a number: ")
8     try:
9         number = int(user_input)
10        if is_palindrome(number):
11            print(f"{number} is a palindrome.")
12        else:
13            print(f"{number} is not a palindrome.")
14    except ValueError:
15        print("Invalid input. Please enter a valid integer.")
16
17
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\kotas\OneDrive\AI assist> & C:/Users/kotas/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/kotas/OneDrive/AI assist/Lab3.1_2256.py"

Enter a number: 13123

13123 is not a palindrome.

PS C:\Users\kotas\OneDrive\AI assist> & C:/Users/kotas/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/kotas/OneDrive/AI assist/Lab3.1_2256.py"

Enter a number: 12121

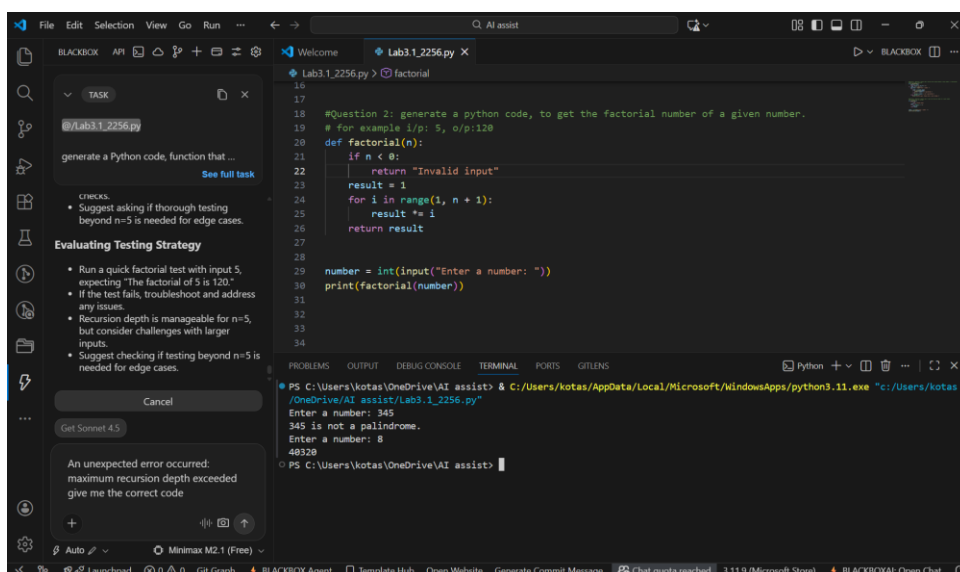
12121 is a palindrome.

PS C:\Users\kotas\OneDrive\AI assist>

- It generates correct logic but does not handle negative numbers explicitly.\

Question 2: One-Shot Prompting (Factorial Calculation)

Prompt: generate a python code, to get the factorial number of a given number.
for example, i/p: 5, o/p:120



```
18
19 #Question 2: generate a python code, to get the factorial number of a given number.
20 # for example i/p: 5, o/p:120
21 def factorial(n):
22     if n < 0:
23         return "Invalid input"
24     result = 1
25     for i in range(1, n + 1):
26         result *= i
27     return result
28
29 number = int(input("Enter a number: "))
30 print(factorial(number))
31
32
33
34
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\kotas\OneDrive\AI assist> & C:/Users/kotas/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/kotas/OneDrive/AI assist/Lab3.1_2256.py"

Enter a number: 345

345 is not a palindrome.

Enter a number: 8

40320

PS C:\Users\kotas\OneDrive\AI assist>

- When compared to zero- short prompting this prompt improves correctness and handles edge cases like zero and negative values.

Question 3: Few-Shot Prompting (Armstrong Number Check)

Prompt: Generate a Python code to check whether a number is an Armstrong number.

Examples: i/p: 153, o/p: Armstrong Number

i/p: 370, o/p: Armstrong Number

i/p: 123, o/p: Not an Armstrong Number

```
28 #Question-3: Generate a Python code to check whether a number is an Armstrong number.
29 #Examples: i/p: 153, o/p: Armstrong Number
30           # i/p: 370, o/p: Armstrong Number
31           # i/p: 123, o/p: Not an Armstrong Number
32 number = int(input("Enter a number: "))
33 print(factorial(number))
34
35 def is_armstrong(num):
36     digits = str(num)
37     power = len(digits)
38     total = 0
39
40     for d in digits:
41         total += int(d) ** power
42
43     return total == num
44
45 number = int(input("Enter a number: "))
46 if is_armstrong(number):
47     print("Armstrong Number")
48 else:
49     print("Not an Armstrong Number")
```

PS C:\Users\kotas\OneDrive\AI assist> & C:/Users/kotas/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/kotas/OneDrive/AI assist/Lab3.1_2256.py"

Enter a number: 1334
1334 is not a palindrome.

Enter a number: 12
479001600

Enter a number: 123
Not an Armstrong Number

- Few-shot prompting improves structural accuracy and produces organized logic.

Question 4: Context-Managed Prompting (Optimized Number Classification)

Prompt: Write an optimized Python program to classify a number as prime, composite, or neither. Ensure input validation and efficient logic.

```
51 #Question-4: Write an optimized Python program to classify a number as prime, composite, or neither.
52 #Ensure input validation and efficient logic.
53 def classify_number(num):
54     if num <= 1:
55         return "Neither Prime nor Composite"
56
57     for i in range(2, int(num ** 0.5) + 1):
58         if num % i == 0:
59             return "Composite Number"
60
61     return "Prime Number"
62
63
64 number = int(input("Enter a number: "))
65 print(classify_number(number))
66
67
```

PS C:\Users\kotas\OneDrive\AI assist> & C:/Users/kotas/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/kotas/OneDrive/AI assist/Lab3.1_2256.py"

Enter a number: 123
Not an Armstrong Number

Enter a number: 233
233 is not a palindrome.

Enter a number: 21
51090942171709440000

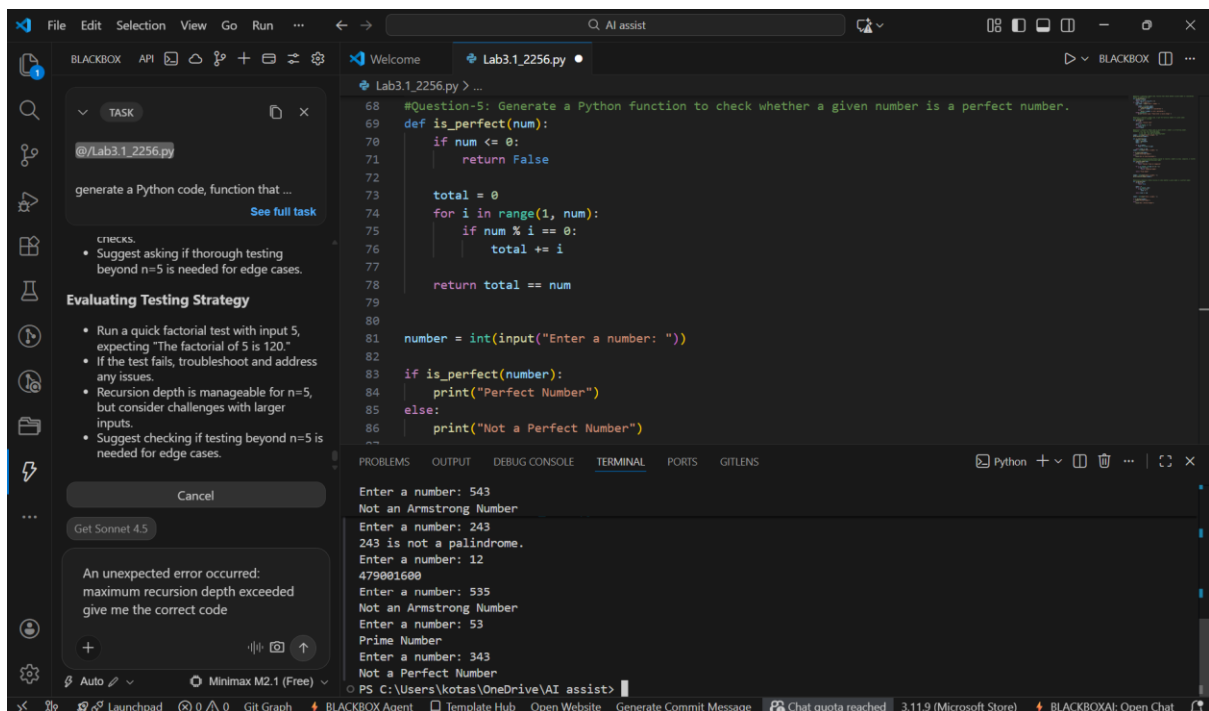
Enter a number: 543
Not an Armstrong Number

Enter a number: 2342
Composite Number

- Context-managed prompts produce optimized and validation-aware solutions suitable for real-world applications.

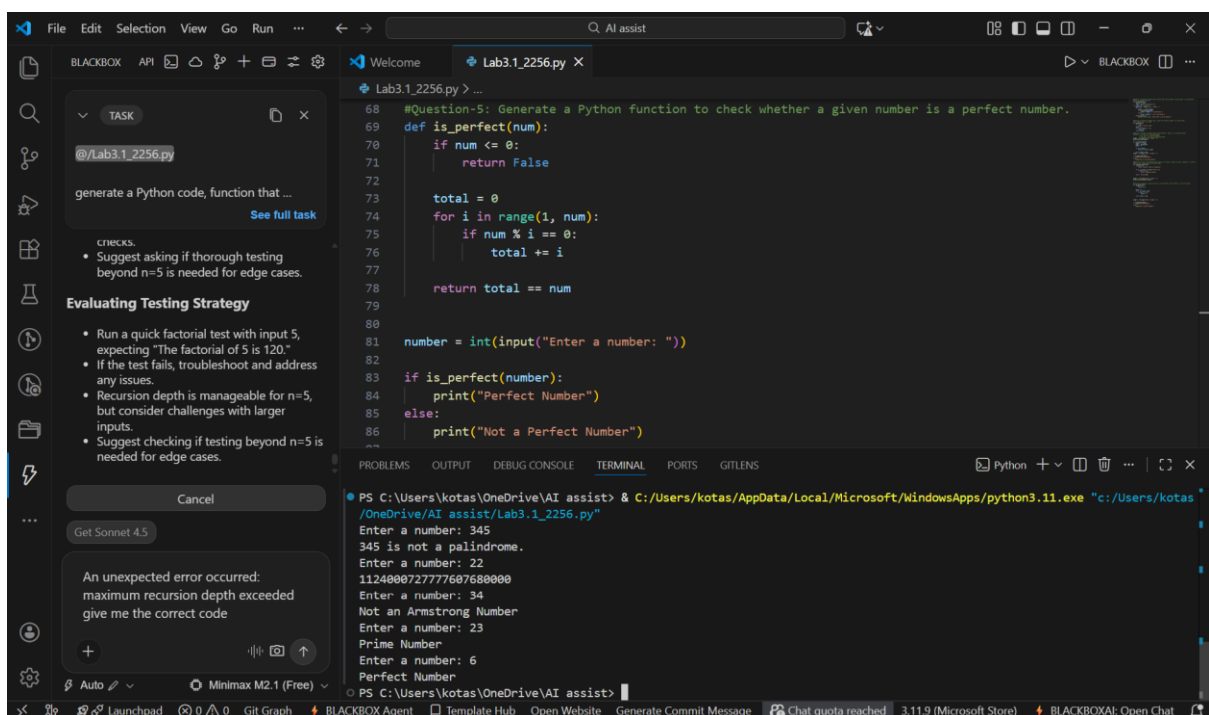
Question 5: Zero-Shot Prompting (Perfect Number Check)

Prompt: Generate a Python function to check whether a given number is a perfect number.



```
68 #Question-5: Generate a Python function to check whether a given number is a perfect number.
69 def is_perfect(num):
70     if num <= 0:
71         return False
72
73     total = 0
74     for i in range(1, num):
75         if num % i == 0:
76             total += i
77
78     return total == num
79
80
81 number = int(input("Enter a number: "))
82
83 if is_perfect(number):
84     print("Perfect Number")
85 else:
86     print("Not a Perfect Number")
```

Enter a number: 543
Not an Armstrong Number
Enter a number: 243
243 is not a palindrome.
Enter a number: 12
479001600
Enter a number: 535
Not an Armstrong Number
Enter a number: 53
Prime Number
Enter a number: 343
Not a Perfect Number



```
68 #Question-5: Generate a Python function to check whether a given number is a perfect number.
69 def is_perfect(num):
70     if num <= 0:
71         return False
72
73     total = 0
74     for i in range(1, num):
75         if num % i == 0:
76             total += i
77
78     return total == num
79
80
81 number = int(input("Enter a number: "))
82
83 if is_perfect(number):
84     print("Perfect Number")
85 else:
86     print("Not a Perfect Number")
```

Enter a number: 345
345 is not a palindrome.
Enter a number: 22
112400072777607680000
Enter a number: 34
Not an Armstrong Number
Enter a number: 23
Prime Number
Enter a number: 6
Perfect Number

- Zero-shot prompting works but is less optimized due to unnecessary full-range iteration.

Question 6: Few-Shot Prompting (Even or Odd Classification with Validation)

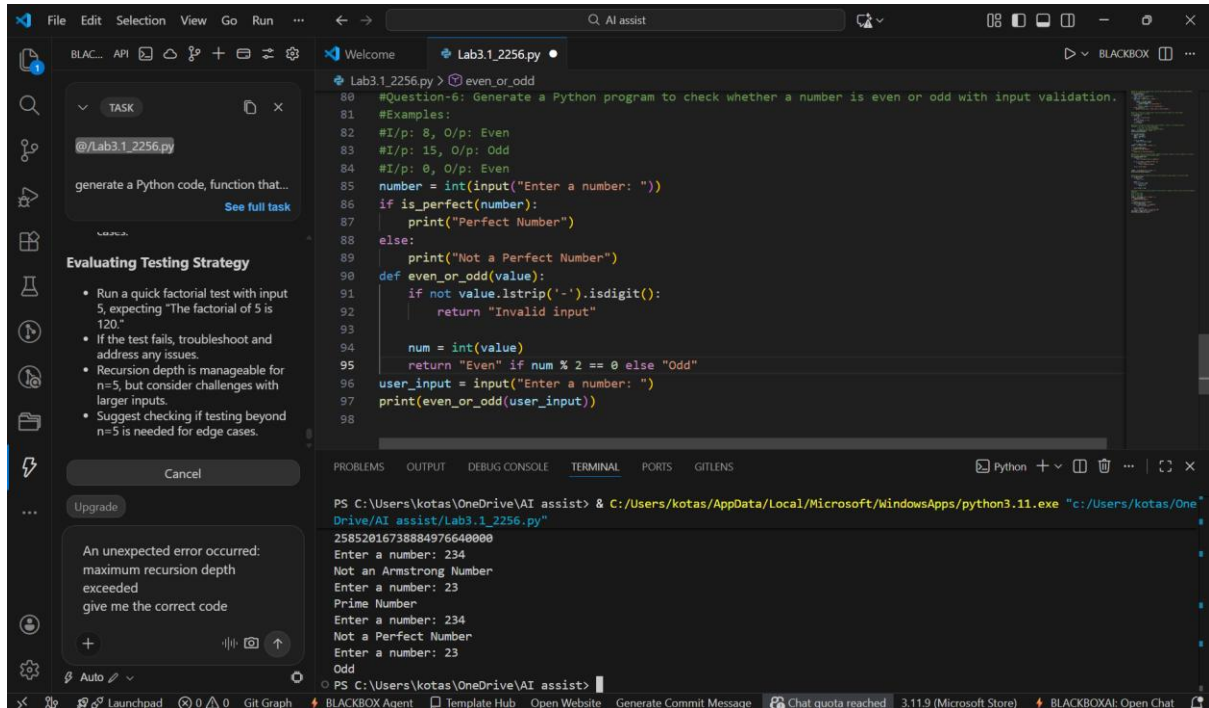
Prompt: Generate a Python program to check whether a number is even or odd with input validation.

Examples:

I/p: 8, O/p: Even

I/p: 15, O/p: Odd

I/p: 0, O/p: Even



```
Lab3.1_2256.py
#Question-6: Generate a Python program to check whether a number is even or odd with input validation.
#Examples:
#I/p: 8, O/p: Even
#I/p: 15, O/p: Odd
#I/p: 0, O/p: Even
number = int(input("Enter a number: "))
if is_perfect(number):
    print("Perfect Number")
else:
    print("Not a Perfect Number")
def even_or_odd(value):
    if not value.lstrip('-').isdigit():
        return "Invalid input"
    num = int(value)
    return "Even" if num % 2 == 0 else "Odd"
user_input = input("Enter a number: ")
print(even_or_odd(user_input))
```

PS C:\Users\kotas\OneDrive\AI assist> C:/Users/kotas/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/kotas/OneDrive/AI assist/Lab3.1_2256.py"

25852016738884976640000

Enter a number: 234

Not an Armstrong Number

Enter a number: 23

Prime Number

Enter a number: 234

Not a Perfect Number

Enter a number: 23

Odd

- Few-shot prompting significantly improves the quality of AI-generated code by providing clear input–output examples.
- The generated program handles input validation effectively, correctly classifies even and odd numbers, and manages negative and non-integer inputs more reliably compared to zero-shot prompting.